



RAPPORT DE STAGE

Sujet : Conception/developpt module entrée/sortie Parquet pour framework
d'apprentissage automatique

LHUILLERY Antonin

Promotion : 2027

Du 2 Septembre 2025 au 30 Janvier 2026

Entreprise : Orange LANNION

Maître de stage :
GOUACHE Stéphane – Ingénieur

Signature :

Table des matières

1	Introduction	2
2	Présentation de l'entreprise	3
2.1	Présentation du groupe	3
2.1.1	Site de Lannion	4
2.2	Présentation de l'équipe	4
3	Déroulement du stage	5
3.1	Travail effectué	5
3.1.1	État de l'art avant le stage	5
3.1.2	Objectifs du projet	5
3.1.3	Ce qui a été produit	6
3.1.4	Points d'appui	6
3.2	Compte rendu d'activité	6
3.2.1	Intégration par conversion préalable en CSV	7
3.2.2	Intégration par lecture du Parquet	10
3.3	Résultats obtenus	12
3.3.1	Intégration par conversion préalable en CSV	12
3.3.2	Intégration par lecture par colonnes	12
3.3.3	Intégration par lecture par groupement de lignes	13
3.3.4	Piste d'amélioration commune à la lecture en temps réel	13
4	Conclusion	14
5	Bibliographie et glossaire	15
5.1	Bibliographie	15
5.2	Glossaire	15
6	Annexes	16

Chapitre 1

Introduction

L'intitulé du sujet de stage est « Conception/développement du module entrée/sortie Parquet pour framework d'apprentissage automatique ». Il consiste à permettre à Khiops, projet open source développé et maintenu par l'entreprise, de prendre en entrée des données au format Parquet. Un format compressé de plus en plus utilisé dans le domaine du Big Data.

Ce stage m'a beaucoup plu car il m'a permis de travailler sur un projet open source, utilisé et maintenu par un grand groupe. J'ai été amené à découvrir un environnement industriel structuré, avec des contraintes de performance, de qualité et de maintenabilité fortes. Avoir de l'expérience dans l'open source est un réel plus en termes d'expérience pour un développeur. De plus, cette entreprise est reconnue à l'international et fait aussi partie des plus grands employeurs du secteur où j'ai grandi. Tout ce dont j'ai entendu parler de cette entreprise est que les conditions y sont très agréables, ce qui a renforcé mon envie de la rejoindre en tant que stagiaire.

J'attendais de ce stage qu'il me permette de mieux comprendre le fonctionnement global de l'entreprise, tant sur le plan organisationnel que technique. J'espérais apprendre beaucoup de nouvelles notions et assimiler les bons réflexes en termes de développement informatique. Je serai très content de pouvoir contribuer à l'outil Khiops en utilisant ce que j'ai appris lors de mes études.

Ce rapport adopte une approche réflexive. Au-delà de la description des tâches réalisées, il vise à analyser les choix techniques effectués, à justifier les orientations prises et à porter un regard critique sur les résultats obtenus. Il permettra également aux personnes amenées à reprendre ce travail d'en comprendre le fonctionnement global et d'identifier les pistes d'amélioration ou d'évolution possibles.

Le déroulement de ce stage peut être découpé en plusieurs étapes. La première a été de prendre en main Khiops et Parquet, qui sont deux nouvelles technologies pour moi. Ensuite, le développement de deux solutions d'intégration, par conversion préalable et par conversion en temps réel.

Chapitre 2

Présentation de l'entreprise

2.1 Présentation du groupe

Orange est un groupe international reconnu dans le secteur des télécommunications et des services numériques. Opérateur historique en France, l'entreprise a su se transformer au fil des décennies pour devenir un acteur global présent dans de nombreux pays et engagé sur des domaines technologiques particulièrement stratégiques. Son identité repose à la fois sur un savoir-faire ancien dans les réseaux et sur une orientation claire vers l'innovation numérique, la donnée et l'intelligence artificielle. Grâce à cette double expertise, Orange occupe aujourd'hui une place centrale dans l'accompagnement de la transformation digitale des particuliers, des entreprises et des organisations publiques. L'entreprise est active sur un large périmètre d'activités : téléphonie mobile, Internet fixe et fibre optique, services TV et multimédias, solutions de cloud computing, cybersécurité, réseaux très haut débit, infrastructure backbone internationale, services financiers mobiles dans certains pays, mais aussi recherche scientifique et innovation appliquée. Cette diversité lui permet d'être un opérateur complet capable de proposer des solutions techniques adaptées à des besoins variés et à des échelles très différentes. Sur un marché particulièrement compétitif, Orange évolue aux côtés d'autres opérateurs majeurs, tels que SFR, Bouygues Telecom et Free sur le territoire français, ou encore d'autres acteurs européens et internationaux à l'étranger. La concurrence se joue tant sur la qualité des infrastructures que sur la rapidité d'innovation et la capacité à répondre aux nouveaux usages numériques. Dans ce contexte, Orange met un accent particulier sur la fiabilité de ses services, la performance de ses réseaux et le développement de solutions innovantes. L'organisation du groupe repose sur un ensemble de directions, services et laboratoires spécialisés, chacun ayant un rôle précis dans la conception, la maintenance, l'évolution et l'innovation des technologies utilisées par l'entreprise. Parmi ces entités, certaines sont dédiées à la recherche et à l'innovation scientifique.

2.1.1 Site de Lannion

Le site Orange de Lannion regroupe près de 1200 salariés autour de différents domaines d’expertise tels que le développement et l’intégration, le commerce conversationnel et le mobile banking, le Big Data et l’IA, la cybersécurité, la virtualisation des réseaux et la 5G, la digitalisation et le service aux entreprises. Anciennement appelé CNET ou France Telecom, ce site est très réputé en France dans le domaine des télécoms. En 2023, le site a fêté ses 60 ans. Ce site a participé à des innovations majeures telles que le Minitel et la Livebox.

2.2 Présentation de l’équipe

C’est dans cette dimension que s’inscrit l’équipe que j’ai intégrée : INNOV/IT-S/DCS/DIF/ADIS, une équipe spécialisée dans la recherche en intelligence artificielle. Cette équipe joue un rôle essentiel dans l’exploration de nouvelles approches, la conception de méthodes algorithmiques avancées et la production de connaissances scientifiques valorisées notamment par la publication d’articles de recherche. Elle participe à la fois à l’avancement de l’état de l’art académique et à l’enrichissement des outils internes du groupe. L’un des projets majeurs porté par l’équipe est le développement de Khiops, un logiciel de data mining reconnu pour sa performance, sa rapidité et son efficacité dans l’analyse de données massives. Cet outil occupe une place importante dans l’écosystème de solutions développées par Orange, car il permet de traiter des enjeux réels liés à la modélisation, à la classification, à la segmentation ou à la prédiction sur de grands volumes de données. En ce sens, l’équipe INNOV/IT-S/DCS se situe à l’intersection entre recherche fondamentale, valorisation scientifique et développement technologique d’outils concrets destinés à être utilisés dans des contextes opérationnels. Son objectif global est d’innover dans le domaine de l’intelligence artificielle, un secteur particulièrement dynamique, en pleine croissance et stratégique pour l’avenir des entreprises numériques. C’est dans ce cadre que s’inscrit mon stage, qui a trouvé une place naturelle dans les travaux de l’équipe en contribuant directement au développement de Khiops. Ma mission consistait à permettre l’utilisation de Khiops avec, en entrée, des données au format Parquet, un format très répandu aujourd’hui dans les environnements big data, notamment grâce à son efficacité de stockage et sa compatibilité avec de nombreux frameworks modernes. L’intégration de ce type de données dans Khiops représente un enjeu important, car elle ouvre la voie à une utilisation plus fluide de l’outil dans des pipelines de traitement contemporains, souvent basés sur des écosystèmes distribués ou des plateformes de données hétérogènes.

Chapitre 3

Déroulement du stage

3.1 Travail effectué

3.1.1 État de l’art avant le stage

Avant le début de mon stage, l’équipe INNOV/IT-S/DCS disposait déjà de l’outil Khiops, un logiciel de data mining et d’intelligence artificielle reconnu pour ses performances et son efficacité dans l’analyse de données de grande dimension. Développé dans un contexte de recherche et largement utilisé dans des environnements professionnels, Khiops permet de construire des modèles prédictifs à partir de données structurées, en automatisant une grande partie des étapes de préparation et de sélection des variables. Khiops est conçu pour répondre à des problématiques industrielles nécessitant le traitement de volumes de données importants, tout en garantissant rapidité d’exécution, robustesse et qualité des résultats. L’un de ses points forts réside dans sa capacité à produire des modèles performants avec un paramétrage simple. L’outil propose également une gestion avancée des données, incluant des mécanismes automatiques de prétraitement, comme la discrétisation des variables numériques, le regroupement des modalités ou encore la sélection des variables pertinentes. Ces fonctionnalités permettent de réduire considérablement le temps consacré à la préparation des données, étape souvent critique dans les projets de data science. Grâce à cette approche, Khiops est particulièrement adapté à des contextes où la fiabilité et la performance sont essentielles. Khiops acceptait en entrée des données sous le format CSV, qui sont des formats de fichiers peu efficace dans le domaine du Big Data cédant progressivement du terrain à des formats compressés. Il n’existait cependant rien auparavant produit dans le projet Khiops en lien avec Parquet.

3.1.2 Objectifs du projet

C’est dans ce contexte que mon stage a vu le jour. L’objectif final du stage est d’étudier le format Parquet pour ensuite permettre son intégration sans modification interne de

Khiops. Les résultats obtenus permettront à l'équipe de mesurer si cette intégration la moins couteuse en termes de temps de développement suffit ou non. Cette demande a été faite par les utilisateurs eux-mêmes. Apache Parquet, étant un format de données très compétitif en termes d'espace de stockage et de lecture des données, est un format complexe mais largement répandu dans la gestion de données.

3.1.3 Ce qui a été produit

J'ai produit deux preuves de conception sur l'intégration de Parquet dans Khiops. Cela implique un outil externe permettant la conversion de données au format Parquet en données au format CSV, avec la création d'un driver permettant la lecture de multi-fichiers. Ainsi que la production d'un autre driver permettant la lecture de fichiers Parquet en temps réel sans conversion préalable. L'objectif de ces deux preuves est d'éclairer l'équipe sur les moyens d'intégration possibles et de permettre de faire le meilleur choix pour répondre à cette demande.

3.1.4 Points d'appui

Pour mener à bien ce stage, j'ai pu bien entendu m'appuyer sur mon maître de stage, Stéphane, ayant une grande connaissance sur Khiops et sur tout ce qui l'entoure. Ainsi que l'équipe qui travaille sur ce projet, ayant chacun leur domaine d'expertise dans chaque partie de ce projet. Lorsque j'avais un doute sur la pertinence d'un choix, l'équipe m'aidait à peser le pour et le contre et, grâce à leur expérience, à éviter de tomber dans des erreurs qu'ils connaissaient. Leur approche était très pédagogique et bienveillante, en veillant toujours à ce que je comprenne les raisons de ces choix. J'ai pu également m'appuyer sur le code de ce projet, qui est public, afin de voir comment les choses étaient faites et pouvoir appliquer au mieux les pratiques de code adoptées. Les librairies Arrow, ayant aussi une librairie Parquet, toutes deux faisant partie d'Apache, permettent la lecture et l'écriture de Parquet ainsi que d'autres types de données. J'ai pu en apprendre un peu plus sur Parquet grâce à des discussions via forum, donnant des solutions aux problèmes d'autres utilisateurs. En effet, internet a constitué un vrai puits d'informations tout au long de cette période, bien qu'elles ne soient pas toujours justes.

3.2 Compte rendu d'activité

Le compte rendu d'activité est écrit de façon à suivre le plus fidèlement possible ce qui s'est passé dans l'ordre chronologique.

Explication Parquet

Une explication préalable du format Parquet est nécessaire afin de bien comprendre tout ce qui va être dit dans cette partie. Comme dit précédemment, Parquet est un format de données moderne qui a été bien réfléchi dans un contexte de Big Data. Sa structure est bien particulière, car il stocke son contenu de manière colonnaire (cf. figure 6.1). Ce choix se justifie par le fait qu'on peut très facilement ignorer des colonnes non pertinentes dans un cas, contrairement aux formats orientés lignes où ce n'est pas possible. Un fichier Parquet comporte plusieurs structures qui regroupent les données (cf. figure 6.2). La plus grande structure est le groupement de lignes, qui sert à diviser le fichier en plusieurs gros morceaux pouvant être gérés en parallèle. Tous ces groupements de lignes sont ensuite constitués de plusieurs colonnes, et les colonnes sont elles-mêmes divisées en pages. Pour obtenir les valeurs, il faut décompresser et décoder les pages. Parquet est très efficace lors de la lecture des données grâce au principe de la lecture sélective. Les structures énoncées précédemment, formant un fichier Parquet, contiennent toutes des métadonnées sur les valeurs de ces structures. Elles donnent accès à des informations telles que la valeur maximale, le nombre de valeurs nulles dans la structure. Cela permet d'éviter la décompression, le décodage et la lecture de structures lorsqu'elles ne nous intéressent pas. Si nous voulons, par exemple, toutes les personnes âgées de plus de 40 ans, la lecture sélective pourra sauter les groupements de lignes pour lequel on sait que l'âge maximal est 20 ans. En plus de son efficacité, il permet de stocker un grand nombre de données dans un volume disque réduit grâce à la compression une fois dans le fichier.

Lors de mon arrivée, il m'a été expliqué qu'il serait préférable de réfléchir à cette intégration par le biais d'un outil externe faisant une conversion préalable de Parquet vers CSV. Sachant que cette méthode serait la plus simple à implémenter, elle nous servirait de référence. De plus, il a été judicieux de la considérer en priorité afin que je puisse comprendre en détail ce format avec toutes ses subtilités.

3.2.1 Intégration par conversion préalable en CSV

Pour m'aider à prendre en main le format Parquet et surtout pour me donner une bonne direction dès le départ, il m'a été conseillé de faire un benchmark sur la lecture et la conversion de ce type de fichier. Les critères sont bien évidemment une conversion efficace, qui ne modifie en aucun cas les données, et rapide, pour ne pas endommager ce qui fait la réputation de Khiops. Un dernier critère, avec lequel je ne m'étais jamais confronté, est que mon code soit efficient au niveau de la mémoire. Dans mon cas, cela signifie que mon code ne doit pas saturer l'espace mémoire de la machine, même pour un fichier infiniment grand.

Un travail de benchmark a été mené afin d'évaluer différentes solutions de conversion selon plusieurs critères essentiels : la fidélité des données converties, les performances en

temps d'exécution, ainsi que la consommation mémoire. Ce dernier point était particulièrement important, l'objectif étant de garantir une conversion capable de traiter des fichiers d'une taille supérieure à la mémoire disponible sur la machine sans la saturer.

Plusieurs approches ont été testées, notamment à travers deux langages (Python et C++) et stratégies de conversion. L'utilisation d'un langage offrant un meilleur contrôle et des bibliothèques plus adaptées à la manipulation du format Parquet s'est rapidement imposée comme une solution plus performante. Des stratégies de parallélisation ont également été explorées afin de réduire le temps de conversion, notamment en séparant les phases de lecture et d'écriture.

Ces expérimentations ont permis de mettre en évidence que l'écriture des fichiers constituait le principal facteur limitant en termes de performance. Une solution basée sur la génération de plusieurs fichiers intermédiaires, considérés comme un ensemble cohérent appelé « multi-fichier », a alors été retenue. Ce que nous appelons un multi-fichier est un ensemble de fichiers qui, mis bout à bout, donnent un seul fichier. La suite du précédent est le début du suivant, ainsi créant cette logique. Cette approche permet d'améliorer significativement les performances en évitant des opérations coûteuses de fusion, tout en garantissant l'intégrité des données.

Afin d'assurer une répartition efficace du travail et une meilleure maîtrise des performances, le découpage des fichiers a finalement été réalisé à partir des groupements de lignes natifs du format Parquet. Nous sommes donc arrivés à une conversion qui surclasse les autres, et nous ne pensons pas pouvoir trouver d'autres méthodes plus performantes (cf. figure 6.3). Après analyse, le temps de conversion de notre base "Accidents" de Parquet vers CSV prend un peu plus de 600 millisecondes (cf. figures 6.5, 6.6, 6.7, 6.8). Sachant que l'exécution d'un entraînement d'un modèle sur cette même base en CSV prend à peu près 9 secondes (cf. figures 6.9). La conversion représente donc 6,67% de l'exécution. C'est avec ces résultats que nous en avons conclu que notre conversion était satisfaisante. Nous en avons donc conclu que le driver était bien fonctionnel et satisfaisant en termes de performance. Ce choix a permis d'obtenir une solution adéquate pour une preuve de concept, avec des temps de conversion compatibles avec les exigences de performance de Khiops.

Permettre de lire les multi-fichiers

Notre conversion produit donc un certain nombre de fichiers qui seront considérés comme un multi-fichier. Nous devons donc permettre à Khiops de lire ce type de fichiers par la création d'un driver. Un driver est une librairie dynamique qui doit implémenter des fonctions telles que l'ouverture d'un fichier, la lecture du fichier une fois ouvert, le déplacement du curseur dans le fichier et la fermeture de ce fichier. Des drivers peuvent aussi permettre l'écriture d'un fichier, mais dans notre cas, un driver n'impliquant que la lecture est suffisant. L'intégration initiale de la conversion directement dans le driver a toutefois

révélé certaines limites, notamment lorsque Khiops ouvrait plusieurs fois les mêmes fichiers lors de certaines fonctionnalités. Après discussion avec l'équipe, il a été décidé de séparer la conversion du mécanisme de lecture. La conversion est ainsi devenue un outil externe, tandis que le driver se concentre uniquement sur la lecture ordonnée des fichiers générés. Cette séparation des responsabilités permet une meilleure robustesse de la solution et une intégration plus cohérente avec l'architecture existante de Khiops. La structure implémentée stocke les noms des fichiers dans un ordre précis, dans notre cas par ordre croissant du numéro dans le nom du fichier. Nous stockons deux positions, la première étant la position du multi-fichier et la seconde étant la position dans le fichier actuellement ouvert (cf. figure 6.4). La mise en place de nombreux tests a permis d'implémenter et de corriger cette implémentation. Une fois que nous savions que notre driver permettait de lire les données sans problème, nous devions nous assurer que ses performances étaient à la hauteur des attentes. Nous avons choisi comme repère, le temps d'exécution de quelques fonctionnalités de Khiops sans utiliser notre driver, et nous obtenons approximativement 12,9 secondes (cf. figure 6.10). Nous obtenons un temps approximatif de 13,1 secondes sur les mêmes fichiers avec notre driver (cf. figure 6.11) et un temps identique pour une lecture avec notre driver sur les fichiers obtenus par la conversion (cf. figure 6.12). Les mesures indiquent que cette solution ne ralentit que très peu l'exécution de Khiops, avec 0,2 secondes de plus avec notre driver. Nous en avons donc conclu que cette intégration était envisageable et répondait à la demande.

Découverte d'un comportement indésirable de Khiops

Afin d'être sûr que cette preuve de concept soit valide, nous devions obtenir des résultats exactement identiques avec l'utilisation ou non de notre driver sur les mêmes données. Après avoir testé ce code afin de m'assurer qu'il fonctionne, j'ai donc essayé d'utiliser Khiops faisant appel à ce driver. Or, les rapports produits avec et sans mon driver étaient différents, bien que les données soient quasi-identiques. La seule différence était que les valeurs non numériques dans le CSV obtenu à partir du Parquet sont toutes entourées de guillemets. Après de nombreux tests avec l'équipe, nous avons eu la certitude que cela venait donc de Khiops. Ce comportement a pu être identifié et remonté à l'équipe.

Amélioration de notre outil de conversion

Notre intégration est coûteuse en espace disque, car il faut avoir l'espace pour stocker les données Parquet sous la forme de données CSV, bien plus volumineuses. Dans le domaine du Big Data, le cloud est largement répandu comme solution à ce problème. C'est pour ces raisons que nous avons essayé de rendre possible l'utilisation de Clouds pour l'outil de conversion. Pour ce faire, j'ai donc implémenté un moteur qui permet

l'écriture grâce à des flux de données héritant des flux propres à Arrow. En faisant cela, je peux écrire et lire avec les fonctions de la librairie Arrow tout en utilisant les flux que j'ai implémentés. Dans l'état actuel des choses, nous pouvons utiliser le disque local et Google Cloud Storage. Il est cependant très simple de rajouter un nouveau moteur pour d'autres Clouds, en s'adaptant aux normes et aux demandes des utilisateurs. Afin de savoir quel type de fichier on veut lire ou écrire, nous suivons le même principe de schéma que les drivers Khiops. Nous résolvons donc le problème majeur de notre intégration actuelle, sans nuire à ses performances. Bien entendu, les écritures et lectures sur Cloud ralentissent le processus, mais pour ce qui est des écritures et lectures en local, les performances restent inchangées. L'état de notre conversion est donc satisfaisant en tant que preuve de conception.

Lors de la présentation de mes travaux à l'équipe en charge du développement de Khiops, nous avons conclu que les objectifs de cette première étape étaient atteints et avons décidé de passer à une intégration sans conversion. La première étape étant de savoir si cela allait être possible ou non.

3.2.2 Intégration par lecture du Parquet

Cette intégration se manifesterait par la création d'un driver Khiops qui, en lisant un fichier Parquet, le traduirait en CSV en temps réel. Cela représente un exercice complexe en raison de la structure du Parquet, à l'opposé de celle du CSV. Jusque-là, j'avais beaucoup appris sur le format Parquet, mais j'y avais très peu touché en profondeur. Cette seconde intégration va me permettre donc d'en apprendre beaucoup plus sur son mode de fonctionnement.

Lecture par colonnes

L'idée qui m'est apparue comme la meilleure était de reconstruire un curseur logique simulant la lecture en lignes. Nous devons donc trouver, pour chaque structure présente dans le Parquet, sa position correspondante orientée en lignes. Avec cela, nous pouvons déterminer dans quel groupement, colonne et ligne se trouve la position de notre curseur. Une fois que nous avons lu toute la valeur, nous allons chercher dans la colonne suivante à la même ligne. Une fois que l'on a fini la ligne, on passe à la ligne suivante. De cette manière, nous lisons un fichier orienté en colonnes de la même manière que s'il était orienté en lignes. La première étape est donc d'être capable d'initialiser notre structure de fichier Parquet. Nous devons donc parcourir entièrement le fichier lors de son ouverture pour obtenir les informations nécessaires. Cela va donc de même pour simplement obtenir la taille du fichier avec le driver, nous devons malheureusement lire entièrement le fichier car il nous faut obtenir sa taille en tant que fichier CSV.

La deuxième étape est de trouver la position de la valeur où se trouve la position

actuelle de notre curseur. Une fois que nous l'avons, nous la lisons grâce au flux de lecture de la colonne. Pour lire une valeur, nous ouvrons le flux de la colonne correspondante et ne sautons pas toutes les valeurs précédentes pour finalement lire celle qui nous intéresse. Il nous reste à implémenter un déplacement logique du curseur à partir de la première valeur lue. Nous devons écrire les valeurs dans un buffer sous la forme d'un CSV. Cela implique les séparateurs ainsi que le retour à la ligne entre respectivement chaque valeur et chaque ligne. Dans notre cas, nous les rajoutons à la fin des valeurs lorsque nous les lisons. Il arrive aussi que nous ayons besoin d'échappement de valeurs. Échapper une valeur dans le contexte du CSV revient à entourer la valeur de guillemets, empêchant une mauvaise compréhension du fichier. Les valeurs nécessitant un échappement sont les séparateurs, les guillemets eux-mêmes ainsi que les retours à la ligne. Les guillemets présents dans la valeur sont quant à eux doublés. Toutes ces particularités sont donc à suivre, et nous devons vérifier si chaque valeur a besoin d'échappement ou non. Cela ralentit le processus, car il implique de parcourir toutes les chaînes de caractères et, dans le cas où l'échappement est nécessaire, la reconstruction de la chaîne échappée. Les en-têtes (headers) sont présents dans les métadonnées et sont donc facilement itérables avant de procéder aux données en elles-mêmes. Ils sont gérés de la même manière que les valeurs, avec une lecture préalable pour obtenir leur taille et initialiser la structure adéquate.

Nous nous retrouvons avec un temps de lecture d'environ 32 secondes pour un fichier de 700 Ko en Parquet et 4,9 Mo en CSV. Ce résultat n'est pas satisfaisant, car bien trop lent pour un petit fichier. Le mécanisme de lecture utilisé est bien trop coûteux pour aucune raison et pourrait grandement s'améliorer. Comme expliqué précédemment, nous ouvrons un flux, le parcourons et le fermons pour la lecture d'une seule valeur. Nous gardons donc les flux de chaque colonne en mémoire. Nous devons donc intervenir dans notre fonction de déplacement de curseur, qui s'assure de garder en mémoire les lecteurs de colonnes du groupement de lignes actuel. Si nous déplaçons le curseur vers l'avant, nous avons qu'à consommer le flux jusqu'au curseur. Or, si nous déplaçons le curseur vers l'arrière, nous devons réouvrir de nouveau ces mêmes flux et les déplacer jusqu'au curseur. Nous obtenons donc un résultat bien meilleur, avec un temps d'environ 4 secondes pour lire le même fichier (cf. figure 6.13). Avec un temps en utilisation avec Khiops d'environ 53 secondes sur un entraînement de modèle, sachant que la même exécution prend 9 secondes (cf. figure 6.14) en lisant l'équivalent en CSV. Cela représente tout de même une différence notable.

Lecture par groupement de lignes

Une seconde approche pour implémenter un driver répondant à nos besoins était de lire par groupements de lignes et de travailler ensuite sur cette structure. Cette version est bien plus simple à implémenter, mais peut se retrouver moins pénalisante qu'on pourrait le penser. En effet, nous lisons un énorme groupement de lignes dont nous n'utiliserons

potentiellement qu’une partie. Mais contrairement à la version précédente, il y a beaucoup moins de calcul, et Parquet est connu pour être efficace quand il s’agit de gros volumes de données. Nous gardons le même principe d’offset logique qui simule la lecture d’un fichier CSV avec les séparateurs, retour à la ligne, échappement, et sans oublier les headers. Le changement va se situer dans la lecture de la valeur. Dans un premier temps, nous identifions dans quel groupement de lignes notre curseur se trouve. Nous lisons ce groupement et utilisons des index de ligne et de colonne pour travailler sur le tableau obtenu. Nous nous déplaçons ensuite à la valeur où se trouve le curseur et commençons la lecture à partir d’ici en incrémentant les indices. Si nous arrivons à la fin du groupement actuel, nous lisons le suivant et réinitialisons les index.

Le temps de lecture de ce même fichier, nous servant de repère, est de 1 seconde (cf. figure 6.15), donc bien mieux que les 4 secondes de la lecture par colonnes. Or, le temps du même entraînement de modèle sur Khiops, avec la même base de données, donne environ 2 minutes et 9 secondes (cf. figure 6.16).

3.3 Résultats obtenus

Les travaux réalisés au cours de ce stage avaient pour objectif principal d’évaluer la faisabilité et la pertinence de différentes stratégies d’intégration du format Parquet dans Khiops sans sa modification interne. Deux approches distinctes ont été explorées : une intégration indirecte par conversion préalable en CSV et une intégration directe par lecture du format Parquet en temps réel.

3.3.1 Intégration par conversion préalable en CSV

La première approche, basée sur une conversion préalable des fichiers Parquet vers des fichiers CSV, s’est révélée être une solution robuste et relativement simple à intégrer. Les benchmarks réalisés ont montré que, moyennant une implémentation adaptée et un découpage pertinent des données, les performances de conversion pouvaient rester compatibles avec les exigences de Khiops.

La mise en place d’un mécanisme de multi-fichiers, accompagnée d’un driver dédié à la lecture du résultat de la conversion, a permis d’assurer une bonne intégration fonctionnelle dans Khiops. Les résultats produits à partir des données converties étant identiques à ceux obtenus avec des fichiers CSV classiques, cette approche peut être considérée comme valide du point de vue de la fiabilité des données.

3.3.2 Intégration par lecture par colonnes

La seconde approche, fondée sur la lecture directe des fichiers Parquet, s’est révélée nettement plus complexe à mettre en œuvre. Elle nécessite en effet de simuler une

lecture orientée lignes à partir d'un format fondamentalement orienté colonnes, ce qui implique une gestion fine des curseurs, des offsets logiques et des mécanismes d'échappement propres au format CSV. Les premières implémentations, basées sur une lecture valeur par valeur via des flux de colonnes, ont mis en évidence des performances insuffisantes, avec des temps de lecture très élevés même pour des fichiers de petite taille. Les optimisations apportées, notamment la conservation des flux de colonnes en mémoire au sein d'un groupement de lignes, ont permis d'améliorer significativement les performances, montrant que certaines limitations initiales étaient davantage liées à la stratégie de lecture qu'au format Parquet lui-même. Cette approche peut encore être travaillée lors de la création de sa structure. En effet, les valeurs en Parquet sont fortement typées et, par découlement, chaque colonne est d'un type bien précis. Les valeurs qui admettent des maximums, comme les entiers sur 32 bits, n'impliquent pas d'être lues pour obtenir leur taille. On peut se dire que tous les entiers de 32 bits vont s'écrire avec le même nombre de caractères que la valeur maximale. Lors de la lecture de la valeur, nous remplirons de '0' non significatifs à gauche. Cette astuce peut nous faire gagner du temps d'exécution, mais dans le pire des cas, des valeurs peuvent donc énormément grossir en termes de nombre de caractères. Je n'ai malheureusement pas eu le temps d'étudier cette piste, qui me semble intéressante.

3.3.3 Intégration par lecture par groupement de lignes

Celle qui consistait à lire les données par groupements de lignes a été plus simple à implémenter. Lors de son analyse, elle s'est montrée plus rapide pour la lecture d'un fichier que la lecture par colonnes, mais beaucoup plus lente lors de son utilisation avec Khiops. Je n'ai malheureusement pas pu trouver la raison de cette incohérence et ne possède aucune piste sur son origine. Je pense cependant que cette intégration possède un vrai potentiel, et trouver la raison de cette différence permettrait de la résoudre.

3.3.4 Piste d'amélioration commune à la lecture en temps réel

La piste qui, selon moi, apporterait le plus de bénéfice est la récupération de la taille du fichier par le driver. Il est indiqué dans la documentation de Parquet que les métadonnées des groupements de lignes et des colonnes contiennent leur taille décompressée. Or, cette variable n'est pas obligatoire, donc de nombreux fichiers ne contiennent finalement pas cette information. Nous sommes donc contraints de parcourir entièrement le fichier Parquet afin de récupérer cette taille. Ce problème est d'autant plus pénalisant que Khiops demande de nombreuses fois cette taille. En effet, récupérer la taille d'un fichier CSV ne nécessite en aucun cas de parcourir ce fichier, et peut être obtenu de manière quasi-instantanée. Lors d'une utilisation standard de Khiops, nous avons passé 41,79 % du temps d'exécution total à calculer la taille du fichier (cf. figure 6.17). Si nous arrivons à limiter ce temps, nous pourrions gagner du temps d'exécution de manière considérable.

Chapitre 4

Conclusion

Ce stage avait pour objectif principal de contribuer à l'évolution de l'outil Khiops par l'intégration du format de données Parquet. Tout au long de cette période, j'ai été amené à découvrir un environnement de travail structuré, exigeant et orienté vers l'innovation, où les choix techniques doivent répondre à des contraintes fortes de performance, de robustesse et de maintenabilité. Mon travail se devait d'être clair, car des personnes vont être amenées à utiliser mon travail pour intégrer Parquet.

Je pense avoir atteint l'objectif qui était fixé en apportant plusieurs solutions afin d'en choisir la plus adéquate. Mon travail s'est basé sur une étude approfondie du format Parquet et de son intégration potentielle dans Khiops. Les deux preuves de conception ont permis d'explorer les avantages et inconvénients de chaque méthode. J'aurais été déçu d'atteindre la fin de cette période en laissant quelque chose de non fonctionnel. Ce sera d'autant plus gratifiant lorsque Khiops sera capable de lire des données au format Parquet, et de savoir que mon travail a contribué à cette nouveauté.

Le bilan est très positif : j'ai pu m'impliquer dans un projet open source reconnu et maintenu par une équipe experte dans ce domaine. Le travail demandé m'a permis d'exercer dans divers domaines tels que la conception, le développement, l'intégration dans un projet fonctionnel, la prise de mesures de performance, l'analyse de technologies et de résultats, ainsi que l'optimisation. Cela m'a permis de donner un sens concret aux années d'études que j'ai effectuées et d'approfondir mes connaissances acquises durant celles-ci. J'ai été accompagné par des personnes compétentes et pédagogues, me laissant évoluer sur ce problème tout en suivant ma progression de manière continue.

Pour finir, ce stage a été une expérience très enrichissante pour moi, que ce soit d'un point de vue technique ou humain. Cela m'a permis de me faire une idée sur le métier de développeur logiciel appliqué dans le domaine du Big Data et de l'IA. Je suis certain qu'elle constituera une base solide pour la suite de mon projet professionnel.

Chapitre 5

Bibliographie et glossaire

5.1 Bibliographie

- Documentation officielle Parquet
- Documentation officielle Arrow
- Documentation officielle Khiops

5.2 Glossaire

open source Code conçu pour être accessible au public : n'importe qui peut voir, modifier et distribuer le code à sa convenance

big data Ensembles de données vastes et variés qui sont volumineuses et dont la taille augmente rapidement au fil du temps

data mining Pratique consistante à rechercher automatiquement de grandes quantités de données afin de découvrir des tendances et des modèles qui vont au-delà de la simple analyse

CSV (Comma-Separated Values) Format de fichier texte simple, orienté lignes, dans lequel les données sont séparées par des délimiteurs (souvent des virgules ou des points-virgules)

benchmark Analyse comparative

IA Intelligence Artificielle

offset logique Position virtuelle représentant l'avancement d'un curseur dans un flux de données

POC / preuve de concept Implémentation expérimentale destinée à valider la faisabilité et la pertinence d'une solution technique avant une intégration définitive.

Chapitre 6

Annexes

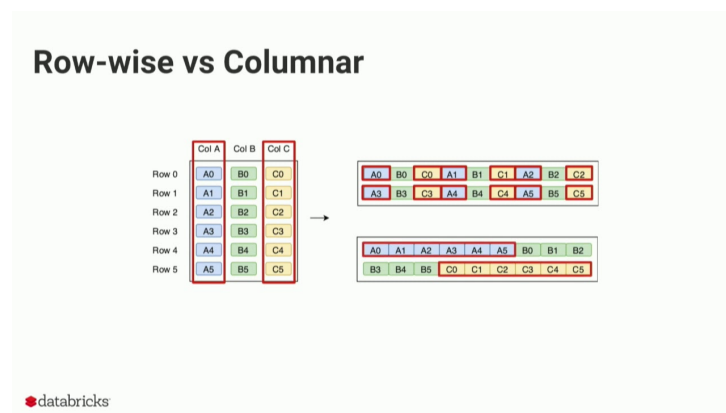


FIGURE 6.1 – Comparaison entre colonnes et lignes orienté

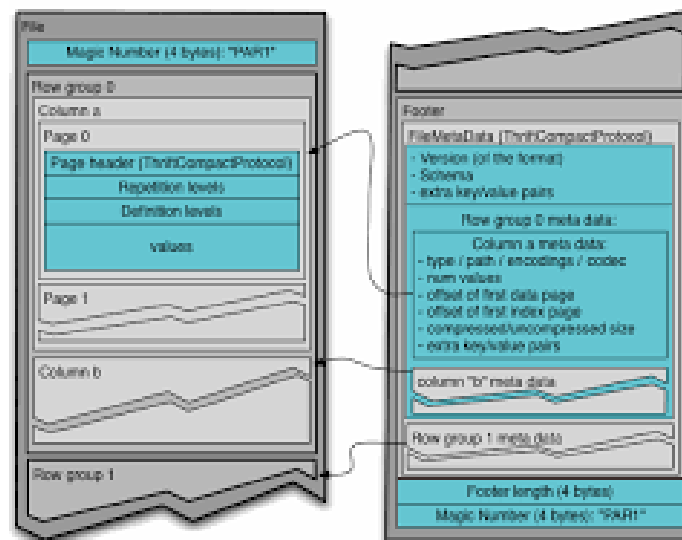


FIGURE 6.2 – Schéma de la structure du format Parquet

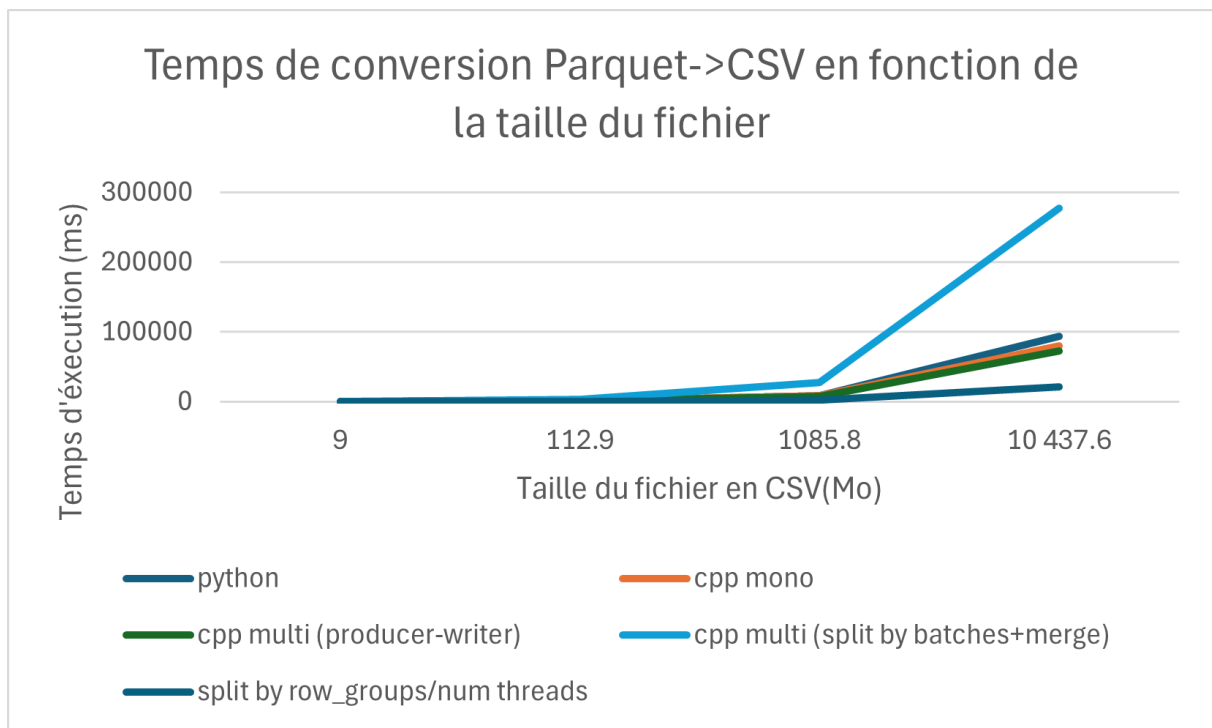


FIGURE 6.3 – Graphique des résultats du benchark

```
// multi-file structure acting as a single large file
struct MultiFile {
    std::string filepath; // original filepath used to open the multifile
    std::vector<std::string> filenames; // list of the mono-files composing the multi-file
    std::vector<uint64_t> file_sizes; // sizes of the mono-files
    std::vector<uint64_t> prefix_offsets; // prefix offsets of the mono-files in the multi-file
    uint64_t total_size = 0;

    size_t current_index = 0; // index of the current file
    FILE* current_handle = nullptr; // handle of the current file
    uint64_t logical_pos = 0; // position in multi-file
    uint64_t pos_in_current = 0; // position in corresponding mono-file

    // MultiFileMode mode = MultiFileMode::READ_ONLY; // for the moment , only read mode is supported
    MultiFileError error_state = MultiFileError::OK;
};
```

FIGURE 6.4 – Structure permettant la lecture de multi-fichiers

```
Total execution time: 71
Conversion succeed: parquet file (C:/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.parquet) saved in csv in
C:/Users/KXFJ3896/Documents/parquet_utils/Accidents/toto*.txt
Output directory not cleaned

C:\Users\KXFJ3896\Documents\parquet_utils\build\windows\parquet_utils_main.exe (processus 4024) s'est terminé avec le co
de 0 (0x0).
```

FIGURE 6.5 – Temps d'exécution de la conversion du fichier Accidents.parquet en CSV

```
Total execution time: 61
Conversion succeed: parquet file (C:/Users/Public/khiops_data/samples/AccidentsMedium/Places.parquet) saved in csv in C:/
Users/KXFJ3896/Documents/parquet_utils/Places/toto*.txt
Output directory not cleaned

C:\Users\KXFJ3896\Documents\parquet_utils\build\windows\parquet_utils_main.exe (processus 6928) s'est terminé avec le co
de 0 (0x0).
```

FIGURE 6.6 – Temps d'exécution de la conversion du fichier Places.parquet en CSV

```
Total execution time: 95
Conversion succeed: parquet file (C:/Users/Public/khiops_data/samples/AccidentsMedium/Vehicles.parquet) saved in csv in C
:/Users/KXFJ3896/Documents/parquet_utils/Vehicles/toto*.txt
Output directory not cleaned

C:\Users\KXFJ3896\Documents\parquet_utils\build\windows\parquet_utils_main.exe (processus 9708) s'est terminé avec le co
de 0 (0x0).
```

FIGURE 6.7 – Temps d'exécution de la conversion du fichier Vehicles.parquet en CSV

```
Total execution time: 397
Conversion succeed: parquet file (C:/Users/Public/khiops_data/samples/AccidentsMedium/Users_medium.parquet) saved in csv
in C:/Users/KXFJ3896/Documents/parquet_utils/Users_medium/toto*.txt
Output directory not cleaned

C:\Users\KXFJ3896\Documents\parquet_utils\build\windows\parquet_utils_main.exe (processus 8848) s'est terminé avec le co
de 0 (0x0).
```

FIGURE 6.8 – Temps d'exécution de la conversion du fichier Users_medium.parquet en CSV

```
Variable construction time: 0:00:00.83 (100 constructed variables)
Train supervised model for classification of target variable Gravity
Database C:\Users\Public\khiops_data\samples\AccidentsMedium\Accidents.txt: Read records: 57,783      Selected
records: 40,561
  Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Places.txt Records: 40,561
  Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Vehicles.txt Records: 69,294
  Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Users_medium.txt Records: 1,431,837
Computation of 10 distinct decision trees
Data preparation time: 0:00:06.16
SNB train time: 0:00:02.17
Build Selective Naive Bayes SNB_Accident
Train evaluation of Selective Naive Bayes on database C:\Users\Public\khiops_data\samples\AccidentsMedium
\Accidents.txt
Test evaluation of Selective Naive Bayes on database C:\Users\Public\khiops_data\samples\AccidentsMedium
\Accidents.txt
Write report C:\Users\Public\khiops_data\samples\AccidentsMedium\AllReports.khj
```

FIGURE 6.9 – Log Khiops d'une exécution d'un entraînement de modèle (train model) sur la base de données Accidents

```

Check database C:\Users\Public\khiops_data\samples\AccidentsMedium\Accidents.txt
Input database C:\Users\Public\khiops_data\samples\AccidentsMedium\Accidents.txt: Read records: 57,783 Selected records: 40,561
Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Places.txt Records: 40,561
Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Vehicles.txt Records: 69,294
Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Users_medium.txt Records: 1,431,837
Checked database C:\Users\Public\khiops_data\samples\AccidentsMedium\Accidents.txt: Checked records: 40,561
Database check time: 0:00:01.01

Deploy model Accident
Input database C:\Users\Public\khiops_data\samples\AccidentsMedium\Accidents.txt: Read records: 57,783
Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Places.txt Records: 57,783
Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Vehicles.txt Records: 98,876
Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Users_medium.txt Records: 1,431,859
Output database C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_Accidents.txt: Written records: 57,783
Table C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_Places.txt Records: 57,783
Table C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_Vehicles.txt Records: 98,876
Table C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_Users_medium.txt Records: 1,431,859
Model deployment time: 0:00:03.97

Variable construction time: 0:00:00.69 (100 constructed variables)
Train supervised model for classification of target variable Gravity
Database C:\Users\Public\khiops_data\samples\AccidentsMedium\Accidents.txt: Read records: 57,783 Selected records: 40,561
Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Places.txt Records: 40,561
Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Vehicles.txt Records: 69,294
Table C:\Users\Public\khiops_data\samples\AccidentsMedium\Users_medium.txt Records: 1,431,837
Computation of 10 distinct decision trees
Data preparation time: 0:00:05.93
SNB train time: 0:00:01.97
Build Selective Naive Bayes SNB_Accident
Train evaluation of Selective Naive Bayes on database C:\Users\Public\khiops_data\samples\AccidentsMedium\Accidents.txt
Test evaluation of Selective Naive Bayes on database C:\Users\Public\khiops_data\samples\AccidentsMedium\Accidents.txt
Write report C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\AllReports.khj

```

FIGURE 6.10 – Log d’une exécution de Khiops avec plusieurs fonctionnalités appelées à la suite (Check database, deploy model et train)

```

Check database multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.txt
Input database multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.txt: Read records: 57,783 Selected records: 40,561
Table multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Places.txt Records: 40,561
Table multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Vehicles.txt Records: 69,294
Table multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Users_medium.txt Records: 1,431,837
Checked database multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.txt: Checked records: 40,561
Database check time: 0:00:00.89

Deploy model Accident
Input database multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.txt: Read records: 57,783
Table multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Places.txt Records: 57,783
Table multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Vehicles.txt Records: 98,876
Table multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Users_medium.txt Records: 1,431,859
Output database C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_multi_Accidents.txt: Written records: 57,783
Table C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_multi_Places.txt Records: 57,783
Table C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_multi_Vehicles.txt Records: 98,876
Table C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_multi_Users_medium.txt Records: 1,431,859
Model deployment time: 0:00:03.90

Variable construction time: 0:00:00.70 (100 constructed variables)
Train supervised model for classification of target variable Gravity
Database multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.txt: Read records: 57,783 Selected records: 40,561
Table multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Places.txt Records: 40,561
Table multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Vehicles.txt Records: 69,294
Table multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Users_medium.txt Records: 1,431,837
Computation of 10 distinct decision trees
Data preparation time: 0:00:05.75
SNB train time: 0:00:01.89
Build Selective Naive Bayes SNB_Accident
Train evaluation of Selective Naive Bayes on database multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.txt
Test evaluation of Selective Naive Bayes on database multifile://C:/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.txt
Write report C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\AllReports.khj

```

FIGURE 6.11 – Log d’une exécution de Khiops avec notre driver multi-fichier sur les mêmes fichiers avec plusieurs fonctionnalités appelées à la suite (Check database, deploy model et train)

```

Check database multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Accidents/toto.txt
Input database multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Accidents/toto.txt: Read records: 57,783      Selected records: 40,561
  Table multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Places/toto.txt Records: 40,561
  Table multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Vehicles/toto.txt Records: 69,294
  Table multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Users_medium/toto.txt Records: 1,431,661
Checked database multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Accidents/toto.txt: Checked records: 40,561
Database check time: 0:00:01.45

Deploy model Accident
Input database multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Accidents/toto.txt: Read records: 57,783
  Table multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Places/toto.txt Records: 57,783
  Table multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Vehicles/toto.txt Records: 98,876
  Table multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Users_medium/toto.txt Records: 1,431,859
Output database C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_multi_slit_Accidents.txt: Written records: 57,783
  Table C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_multi_slit_Places.txt Records: 57,783
  Table C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_multi_slit_Vehicles.txt Records: 98,876
  Table C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\T_multi_slit_Users_medium.txt Records: 1,431,859
Model deployment time: 0:00:04.08

Variable construction time: 0:00:00.84 (100 constructed variables)
Train supervised model for classification of target variable Gravity
Database multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Accidents/toto.txt: Read records: 57,783      Selected records: 40,561
  Table multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Places/toto.txt Records: 40,561
  Table multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Vehicles/toto.txt Records: 69,294
  Table multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Users_medium/toto.txt Records: 1,431,661
Computation of 10 distinct decision trees
Data preparation time: 0:00:05.81
SNB train time: 0:00:01.79
Build Selective Naive Bayes SNB_Accident
Train evaluation of Selective Naive Bayes on database multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Accidents/toto.txt
Test evaluation of Selective Naive Bayes on database multifile://C:/Users/KXFJ3896/Documents/Parquet-Integration/parquet_utils/Accidents/toto.txt
Write report C:\Users\KXFJ3896\Documents\Parquet-Integration\test_driver\cd_dm_train\T_files\AllReports.khj

```

FIGURE 6.12 – Log d’une exécution de Khiops avec notre driver multi-fichier sur les fichiers découpés équivalent avec plusieurs fonctionnalités appelées à la suite (Check database, deploy model et train)

```

File logical size: 7873347
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 533315
Parquet reading succeed in 3934millisecs
Total = 7873347

C:\Users\KXFJ3896\Documents\parquet_reader\build\windows-release\parquet_reader_main.exe (processus 8940) s'est terminé
avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .

```

FIGURE 6.13 – Temps de lecture du fichier Accidents.parquet dans la base Accidents avec la lecture par colonne

```

Variable construction time: 0:00:32.23 (100 constructed variables)
Train supervised model for classification of target variable Gravity
Database parquet://C:/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.parquet: Read records: 57,783
  Selected records: 40,561
  Table parquet://C:/Users/Public/khiops_data/samples/AccidentsMedium/Places.parquet Records: 40,561
  Table parquet://C:/Users/Public/khiops_data/samples/AccidentsMedium/Vehicles.parquet Records: 69,294
  Table parquet://C:/Users/Public/khiops_data/samples/AccidentsMedium/Users_medium.parquet Records: 1,431,782
Computation of 10 distinct decision trees
Data preparation time: 0:00:19.06
SNB train time: 0:00:01.99
Build Selective Naive Bayes SNB_Accident
Train evaluation of Selective Naive Bayes on database
parquet://C:/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.parquet
Test evaluation of Selective Naive Bayes on database
parquet://C:/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.parquet
Write report C:\Users\KXFJ3896\Documents\parquet_reader\scenarios\cd_dm_train\T_files\AllReports.khj

```

FIGURE 6.14 – Log d’une exécution de Khiops avec notre driver lisant en temps réel le Parquet par colonnes

```

File logical size: 7873347
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 1048576
Read bytes = 533315
Parquet reading succeed in 997millisecs

C:\Users\KXFJ3896\Documents\parquet_reader\build\windows-release\parquet_reader_main.exe (processus 11416) s'est terminé
avec le code 0 (0x0).

```

FIGURE 6.15 – Temps de lecture du fichier Accidents.parquet dans la base Accidents avec la lecture par groupements de lignes

```

Variable construction time: 0:01:25.63 (100 constructed variables)
Train supervised model for classification of target variable Gravity
Database parquet://C/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.parquet: Read records: 57,783
Selected records: 40,561
Table parquet://C/Users/Public/khiops_data/samples/AccidentsMedium/Places.parquet Records: 40,561
Table parquet://C/Users/Public/khiops_data/samples/AccidentsMedium/Vehicles.parquet Records: 69,294
Table parquet://C/Users/Public/khiops_data/samples/AccidentsMedium/Users_medium.parquet Records: 1,431,782
Computation of 10 distinct decision trees
Data preparation time: 0:00:41.92
SNB train time: 0:00:01.85
Build Selective Naive Bayes SNB_Accident
Train evaluation of Selective Naive Bayes on database
parquet://C/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.parquet
Test evaluation of Selective Naive Bayes on database
parquet://C/Users/Public/khiops_data/samples/AccidentsMedium/Accidents.parquet
Write report C:\Users\KXFJ3896\Documents\parquet_reader\scenarios\cd_dm_train\T_files\AllReports.khj

```

FIGURE 6.16 – Log d’une exécution de Khiops avec notre driver lisant en temps réel le Parquet par groupements de lignes

```

driver_getFileSize
  Appels : 121
  Total (ms) : 118302
  Portion du total : 41.79 %
  Moyenne (ms) : 977.70
  Variance : 1970516.89
  Écart-type : 1403.75
  Min / Max : 110 / 3914

driver_fopen
  Appels : 111
  Total (ms) : 110369
  Portion du total : 38.99 %
  Moyenne (ms) : 994.32
  Variance : 2027956.42
  Écart-type : 1424.06
  Min / Max : 121 / 3804

driver_fseek
  Appels : 437
  Total (ms) : 305
  Portion du total : 0.11 %
  Moyenne (ms) : 0.70
  Variance : 0.93
  Écart-type : 0.97
  Min / Max : 0 / 6

driver_fread
  Appels : 594
  Total (ms) : 51945
  Portion du total : 18.35 %
  Moyenne (ms) : 87.45
  Variance : 12296.04
  Écart-type : 110.89
  Min / Max : 2 / 566

driver_fclose
  Appels : 111
  Total (ms) : 2168
  Portion du total : 0.77 %
  Moyenne (ms) : 19.53
  Variance : 833.25
  Écart-type : 28.87
  Min / Max : 0 / 87

Total (ms) : 283089

```

FIGURE 6.17 – Analyse des différentes fonctions appelées de notre lecteur de parquet par colonnes sur une exécution standard de Khiops